INDEX

NAME: Deva Dharshini P STD.: _____ SEC.: _____ ROLL NO.: _____ SUB.: Computer Network Observation

S. No.	Date	Title	Page No.	Teacher's Sign / Remarks
01.	20/07/26	Study of various Network commands used in Linux and windows	9	24/8/26
02.	28/07/26	Study of different types of network cables.	9	4/8/26
03	3/8/26	CISCO PACKET TRACER	9	10/8
04	10/8/26	Setup and configure a LAN using switch and ethernet cables	10	18/8
05	18/8/26	WIRESHARK	9	7/9
06	7/9/26	Error Detection & correction using hamming code	9	24/9
07	22/9/26	NMAP to discover live hosts using NMap scans on the TryHackme platform	10	28/9
08	28/9/26	End to End communication at transport layer	10	29/9
09	28/9/26	Sliding window protocol	9	29/9

S. No.	Date	Title	Page No.	Teacher's Sign / Remarks
10	6/10/26	Implementation of subnetting in cisco packet Tracer simulator		
11	6/10/26	Implement your own ping program		
12	13/10/26	Packet sniffing using raw sockets		
13	13/10/26	Internetworking with routers in cisco packet Tracer		
14.	13/10/26	Routing Information protocol.		

Ex no: 1
20/7/26

Study of various Network commands used in Linux and windows

Aim:

To study of various Network commands used in Linux and windows.

Basic Networking commands:

arp-a:

ARP is short form of address resolution protocol It will show the IP address of your computer along with the IP address and MAC address of your router.

hostname:

This is the simplest of all TCP/IP commands. It simply displays the name of your computer.

ipconfig/all:

This command displays detailed configuration information about your TCP/IP connection including Router, Gateway, DNS, DHCP and type of Ethernet adapter in your system.

nbstat -a:

This command helps solve problems with NetBIOS name resolution.

netstat:

(network statistics) netstat displays a variety of statistics about a computers active TCP/IP connections. It is a command line tool for monitoring network connections both incoming and

outgoing as well as viewing routing tables, interface statistics, etc.

eg: netstat -r

nslookup:

(name server lookup) is a tool used to perform DNS lookups in Linux. It is used to display DNS details, such as the IP address of a particular computer, the MX records for a domain or the NS servers of a domain. nslookup can operate in two modes: interactive and non-interactive.

eg: nslookup www.google.com

pathping:

Pathping is unique to Windows and is basically a combination of the Ping and Tracert commands. Pathping traces the route to the destination address then launches a 25 second test of each router along the way, gathering statistics on the rate of data loss along each hop.

ping:

(packet INternet Groper) command is the best way to test connectivity between two nodes. Ping use ICMP (Internet Control Message Protocol) to communicate to other devices.

1. #ping hostname (ping localhost)
2. #ping ip address (ping 4.2.2.2)
3. #ping fully qualified domain page (ping www.facebook.com)

Route:

Route command is used to show/manipulate the IP routing table. It is primarily used to

setup static routes to specific host or networks via an interface.

Some important Linux networking commands

1 ip

The ip command is one of the basic commands every administrator will need in daily work and assigning IPs to troubleshooting existing systems. It shows address information, manipulate routing, interfaces and tunnels.

ip <options> <object> <command>

a) To show the IP address assigned to an interface on your server:

[root@server~] # ip address show

b) To assign an IP to an interface,

[root@server~]# ip address add 192.168.254.2 dev enp0s3

c) Alter the status of the interface by bringing the interface

[root@server~]# ip link set etho up

d) Alter the status of the interface by bringing the interface

[root@server ~]# ip link set etho down

e) Delete the route

[route @ server~] # ip route delete

2 ifconfig

The ifconfig command was/is a staple in many sys admin's tool belt for configuring and troubleshooting networks. It has since been replaced by the ip command discussed above

3. mtr

(Matt's traceroute) is a program with a command-line interface that serves as a network diagnostic and troubleshooting tool. This command combines the functionality of the ping and traceroute commands.

The syntax of the command is as follows:

mtr <options> hostname/IP

a) The basic mtr command shows statistics

[root@server ~]# mtr google.com

b) Show numeric IP addresses

[root@server ~]# mtr -g google.com

c) Show the numeric IP addresses and hostnames.

[root@server ~]# mtr -b google.com

d) Set the number of pings that you want to send

[root@server ~]# mtr -C 10 google.com

4. tcpdump

The tcpdump command is designed for capturing and displaying packets. It helps troubleshoot problems.

eg: tcpdump -i eth0

5. Ping

Ping is a network utility used to check connectivity between two devices on a network. It sends ICMP Echo Request messages to a destination host and waits for Echo Reply messages.

eg:
ping google.com

ping 10 times:
ping -c 10 google.com

- Reply Received: Connection is working properly
- Destination Host Unreachable: No route to the destination host.
- Request Timed out: No response received from the host
- Unknown Host: Hostname is incorrect or cannot be resolved.

* good ping result should have low latency and 0% packet loss.

Result:
The various Network commands used in Linux and windows are executed successfully.

27/7/26

Ex no: 2 Study of different types of Network Cables
28/7/26

Aim:

To study of different types of Network cables.

a.) Understand different types of network cable

Different type of cables used in networking are:

1. Unshielded Twisted Pair (UTP) cable
2. Shielded Twisted Pair (STP) cable
3. Coaxial cable
4. Fibre optic cable

Cable Type	Category	Maximum Data Transmission	Advantages / Disadvantages	Application / Use
UTP	Category 3	10 bps	Advantages • Cheaper in cost • Easy to install as they have a smaller overall diameter. Disadvantages • More prone to (EMI) Electromagnetic interference and noise.	10 Base-T Ethernet
	Category 5	Upto 100 Mbps		Fast Ethernet, Gigabit Ethernet
	Category 5e	1 Gbps		Fast Ethernet, Gigabit Ethernet
STP	Category 6, 6a	10 Gbps	Advantages • Shielded • Faster than UTP • Less susceptible to noise and interference	Gigabit Ethernet, 10G Ethernet (55m) Widely used in data centres
SSTP	Category 7	10 Gbps		

			Disadvantages • Expensive • Greater installation effort	Gigabit Ethernet, 10 Gb Ethernet (100 m)
Coaxial Cable	RG-6 RG-59 RG-11	10-100 Mbps	Advantages • High bandwidth • Immune to interference • Low loss bandwidth • Versatile Disadvantages • Limited distance • Cost • Size is bulky	Speed of signal is 500m Television network High speed internet connections
fibre optics cable	Single mode Multimode	100Gbps	Advantages • High speed • High bandwidth • High security • Long distance Disadvantages • Expensive • Requires skilled installers.	Maximum distance of fibre optics cable is around 100 meters

Result:

Thus, the different types of Network cables are studied successfully.

Exp no: 3 Cisco Packet Tracer
9/8/26

Aim:

To study cisco packet Tracer Tool installation and user interface overview.

Introduction:

A simulator, as the name suggests, simulate network devices and its environment. Packet Tracer is an existing network design, simulation and modelling tool.

* It allows you to model complex systems without the need for dedicated equipment.

* It helps you to practice your network configuration and trouble shooting skills via computer or an Android or iOS based mobile devices.

* It is available for both the Linux and windows desktop environments.

* Protocols in Packet Tracer are coded to work and behave in the same way as they would on real hardware.

→ Analyse the behaviour of network devices using Cisco Packet Tracer simulator.

i.) Peer to Peer

ii.) Switch

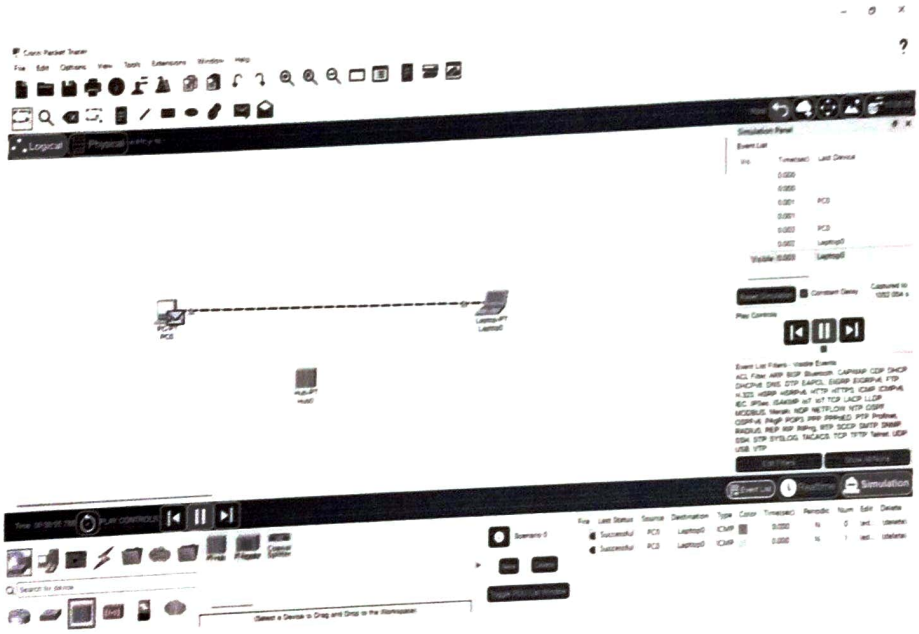
iii.) Hub

i) Peer to Peer

A peer to peer network is a network in which computers communicate directly with each other without requiring a dedicated central server.

Eg:

Two PC's connected directly using a cable and communicating with each other.



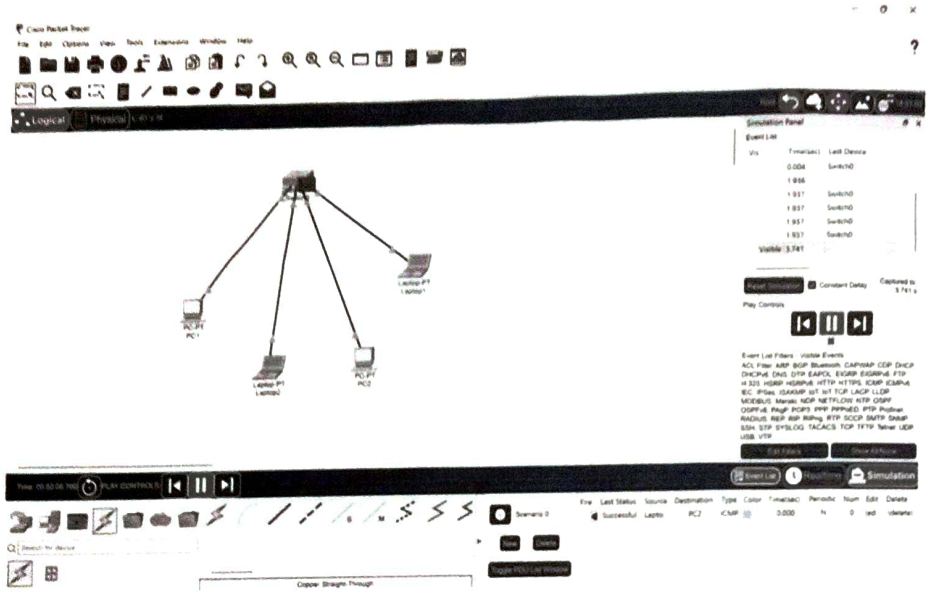
ii) Switch

A switch is a networking device used to connect multiple devices in a network and forward data only to the intended destination device.

Eg:

PC1 → switch → PC3

Switch knows where PC3 is and forwards the frame only toward PC3. Switch only works at the Data link layer (layer 2) of the OSI model. It was MAC address sends data to the required port.

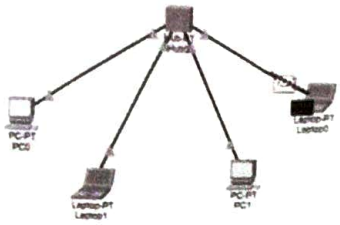


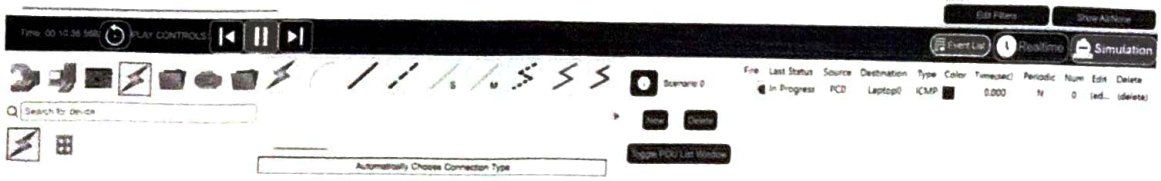
iii) Hub:

A Hub is a basic networking device used to connect multiple computers in a network. It receives data from one device and broadcasts it to all connected devices. Hub works mainly at Physical layer (layer 1) of the OSI model.

Eg:

If PC1 sends data to PC3 through a hub, the hub sends that data to PC2, PC3, PC4 etc.







Result:

Thus, the cisco packet Tracer Tool is studied and executed sucessfully.

Ex no: 4 Setup and Configure a LAN using switch and Ethernet cables
10/8/26

What is LAN?

LAN stands for Local Area Network which is a group of computers and connected devices linked together in a small physical location like a home, school or office building.

How to setup LAN

step 1 : Connect the Hardware

→ place your router in a central location near a power outlet.

→ connect your modem to the router's WAN or Internet port using an Ethernet cable.

→ plug the router into power and wait for it to turn on.

step 2: Connect your devices

→ wired setup : plug one end of an Ethernet cable into your computer's networks port and the other end into one of the numbered LAN ports on the router.

step 3: Configure the router

→ Type your IP address (192.168.1.1 or 192.168.0.1) into the address bar and press enter.

→ Login using the default username and password printed on the router.

step 4: Check the connectivity between switch and other machine by using ping command in the command prompt of the device.

step 5: Try to access the shared folder from others computers of the network

Result:

Wireless LAN using switch and ethernet cables are executed successfully in also cisco packet Tracer.

Ex no: 5 WIRESHARK

18/8/26

Aim:

Experiments on packet capture tool: wireshark

To filter, capture, view, packets in wireshark Tool. Capture 100 packets from the Ethernet: IEEE 802.3 LAN interface and save it.

Procedure:

* Select local Area Connection in Wireshark.
* To capture option
* Select stop capture automatically after 100 packets.
* Then click, start capture
* Search TCP packets in search bar.
* To see flow graph click satisfies flow graph.
* Save the packets.

1. Create a Filter to display only TCP/UDP packets, inspects the packets and provide the flow graph

* Select LAN connection in wireshark.
* To capture option
* Select stop capture automatically after 100 packets
* Then click start capture
* Save the packets

2. Create a Filter to display only ARP packets and inspect the packets procedure:

* Go to capture option
* Select stop capture automatically after 100 packets

* Then click start capture.
* Search ARP packets in search bar
* Save the packets

3. Create a Filter to display only HTTP packets and inspect the packets procedure
* Select local Area Connection in wireshark.
* Go to capture option
* Select stop capture automatically after 100 packets.
* Then click start capture.
* Search HTTP packets in search bar.

4. Create a file to display only DNS packets and provide the flow graph.

Procedure:-

* Go to capture option
* Select stop capture automatically after 100 packets
* Then click start capture.
* Search DNS packets in search bar
* To see the flow graph click statistics flow graph.

5. Create a Filter to display only DHCP packets and inspect the packets.

* Select local Area Connection in wireshark.
* Go to capture option
* Select stop capture automatically after 100 packets.
* Then click start capture.

* Search DHCP packets in search bar.
* Save the packets.

Result:

Wireshark on packet capture tool experiment are learned and executed successfully.

Ex no: 6 Error detection & correction using Hamming Code

7/9/26

Aim:

To write a program to implement error detection and correction using hamming code concept. Make a test run to input data stream and verify error correction feature.

Sender:

* Input to sender file should be a test of any length program should be convert the text to binary.
* Apply hamming code concept on the binary data and add redundant bits to it.
* Save this output in a file called channel.

Code:

```
data = input("Enter 4 bits: ")
d1, d2, d3, d4 = map(int, data)
p1 = d1 ^ d2 ^ d4
p2 = d1 ^ d3 ^ d4
p3 = d1 ^ d3 ^ d4
code = [p1, p2, d1, p4, d2, d3, d4]
print("Sender Hamming Code:", "".join
      (map(str, code))).
```

Receiver side:

* Receive the hamming code from the sender.
* Enter the received code.
* Calculate the parity check values

* Find the error position using the parity values
* Correct the error by changing the bit.
* Display the corrected hamming code.

Code:

```
r = list(map(int, input("Enter received 7 bits: ")))
r1 = r[0]^r[2]^r[4]^r[6]
r2 = r[1]^r[2]^r[5]^r[6]
r3 = r[3]^r[4]^r[5]^r[6]
error = r1*1 + r2*2 + r4*4

if error == 0:
    print("No error")
else:
    print("Error at position:", error)
    r[error-1] ^= 1
    print("Corrected code:", "".join(map(str, received)))
```

Input 1:

Enter 4 data bits: 1011
Sender Hamming Code: 0110011
Received 7 bits: 0010011

Output:

Error at position: 2
Corrected code: 0110011

Input 2:

Enter 4 data bits: 0010
Sender Hamming Code: 0101010
Enter received 7 bits: 0101010

Output:

No error

Result:

Error detection and Error correction using Hamming code was executed successfully.

Ex no: 7 NMAP to Discover live Hosts using NMAP scans on the TryHackme platform
22/9/26

Aim:

To Discover live Hosts using NMAP scans (ARP, ICMP, TCP/UDP) on the TryHackme platform.
Room link; https://tryhackme.com/room/nmapol

Procedure:

Task 1: Introduction

* ARP scan: this scan uses ARP requests to discover live hosts.
* ICMP scan: this scan uses ICMP requests to identify live hosts.
* TCP/UDP ping scan: this scan sends packets to TCP ports and UDP ports to determine live hosts

Task 2: Subnetworks

Network segment - It is a collection of computers linked together through a common "medium".
Subnetworks - It is usually the equivalent of one or more networks segments linked together and configured to utilize the same router in an IP networks.

* Subnets with /16
* Subnets with /24

Task 3: Enumerating Targets

We need to identify the targets we want to scan before we conduct any 'action'.

1. list
- <MACHINE-IP> <scanme.nmap.org> <example.com>
- will scan 3 IP addresses

2. range
- 10.11.12.15-20
- will scan 6 IP addresses

3. subnet
- <MACHINE-IP>/30
- will scan 4 IP addresses

nmap -SL <TARGETS>

Task 4 : Discovering live Hosts

* ARP from link layer - commonly used for Man-In-middle-Attack (ARP spoofing and ARP poisoning)
* ICMP from Network layer - commonly used in DDOS attacks
* TCP from Transport layer - SYN flooding, TCP reset, TCP session hijacking attacks.
* UDP from Transport layer - Also used in DDOS attacks.

Task 5: Nmap Host Discovery using ARP

* When a privileged user tries to scan LAN (Local Area Network), Nmap uses ARP requests.
* To discover internet hosts without port scanning live systems are nmap -sn Targets.

* nmap - PR - SN <TARGETS>

-PR specifies that you only want an ARP scan.

* nmap -PR -SN <MACHINE IP>/24

* nmap -PR -SN <ATTACK BOX IP>/24

Task 6: Nmap host discovery using ICMP

* Add -PE option to utilize ICMP echo requests to discover live hosts.

* nmap -PE -SN TARGET to check host is live

* -PP option instructs Nmap to use ICMP timestamp requests.

nmap -PP -SN TARGET

* -PM option can enable this scan to ICMP address mark requests.

nmap -PM -SN TARGET

Task 7: Nmap Host discovery using TCP and UDP.

* Nmap to use TCP SYN ping use -PS followed by the port number, range, list or a combination of this options.

-PS21, -PS25 targets ports 21, 22, 23, 24, 25

* nmap -PS -SN TARGET check the TCP port is open.

* TCP ACK ping - you must be running Nmap as a privileged user.

* nmap -PA -SN TARGET to discover the online hosts on the targets subnet.
* UDP ping - sending a UDP packet to an open port is unlikely to result in a response.
* If we transmit a UDP packet to a closed UDP port, we should expect to receive an ICMP port unreachable packet.

nmap -PU -SN TARGET

Task 8: Using Reverse-DNS lookup

* Nmap is to use reverse-DNS internet hosts.
* -R option to query the DNS server even for offline hosts.

Task 9: Summary

ARP scan, ICMP Echo scan, ICMP Timestamp scan, ICMP address mark scan, TCP SYN ping scan, TCP ACK ping scan, UDP ping scan.

Result:

NMAP to discover live host using tryhackme platform was sucessfully executed.

Ex no: 8 End to End Communication at Transport Layer

28/9/24

a) Aim:

To implement echo client server using TCP/UDP packets.

Procedure:

* Create a socket using python socket programming.
* Specify the IP address and port numbers.
* Bind the server socket to the IP address and port.
* Make the server listen for client connections.
* Connect the client to the server
* Send and receive messages between client and server.
* Display the received message
* Close the socket connection

Code:

TCP-server:

```
import socket
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind("127.0.0.1", 5000))
server.listen()
print("Server is waiting for connection...")
conn, addr = server.accept()
print("Client connected")
```

```
while True :
    data = conn.recv(1024).decode() : mi A
    if not data :
        break
    print("client:", data)
    conn.send(data.encode())
    conn.close()
server.close()
```

TCP-Receiver (client)

```
import socket
client = socket.socket(socket.AF_INET, socket.
                                SOCK_STREAM)
client.connect(("127.0.0.1", 5000))
message = input("Enter message:")
client.send(message.encode())
reply = client.recv(1024).decode()
print("server:", reply)
client.close()
```

Terminal 1 - server

```
python tcp-server.py
server is waiting for
connection...
client connected
client: Hello
```

Terminal 2 - client

```
python tcp-client.py
Enter message: Hello
server: Hello
TCP echo is working
```

Result:

The TCP/UDP packets using echo client server socket program is executed successfully.

b.)

Aim:

To implement chat client server using TCP packets.

Code:

chat - server side

```
import socket
server = socket.socket(socket.AF_INET, socket.
                       SOCKET_STREAM)
server.bind(("127.0.0.1", 6000))
server.listen(1)
print("waiting for client...")
conn, addr = server.accept()
print("client connected")
while True:
    message = conn.recv(1024).decode()
    if message.lower() == "bye":
    print("client ended chat")
    break
    print("client:", message)
    reply = input("server:")
    conn.send(reply.encode())
    if reply.lower() == "bye":
conn.close()
server.close()
```

chat: client side

```
import socket
client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect(("127.0.0.1", 6000))
print("connected to server")
while True:
    message = input("client:")
    client.send(message.encode())
    if message.lower() == "bye":
        break
    reply = client.recv(1024).decode()
    print("server:", reply)
    if reply.lower() == "bye":
        break
client.close()
```

Terminal 1

```
python chat_server.py
waiting for client...
client connected
client: Hello
server: Hi
client: How are you?
server: Fine
```

Terminal 2

```
python chat_client.py
connected to server
client: Hello
server: Hi
client: How are you?
server: Fine
```

Result:

The chat programming using sockets by TCP client & server was implemented successfully.

28/2/26

Ex no: 9 SLIDING WINDOW PROTOCOL
28/9/26

Aim:

To write a program to implement flow control at data link layer using sliding window protocol. Simulate the flows from one node to another.

Code:

```
import time
window = int(input("Enter window size:"))
message = input("Enter message:")
start = 0
while start < len(message)
print("SENDER")
end = start + window
if end > len(message):
    end = len(message)
# sender buffer
sender_buffer = []
for i in range(start, end):
    frame = (i, message[i])
    sender_buffer.append(frame)
    print("Frame No:", i, "DATA:", message[i])
print("Sender Buffer:")
print(sender_buffer)
#Delay
print("\nWaiting for receiver...")
time.sleep()
# Receiver
print("RECEIVER")
expected = start
error = False
for frame in sender_buffer:
```

```
frame_no = frame[0]
data = frame[1]
print("Received Frame:", frame_no, "DATA:", data)
if frame_no == expected:
    expected = expected + 1
else:
    error = True
    break
#Receiver buffer
if error:
    receiver_buffer = ("NACK", expected)
    print("\n Receiver Buffer:")
    print(receiver_buffer)
else:
    ack = expected - 1
    receiver_buffer = ("ACK", ack)
    print("\n Receiver buffer:")
    print("receiver_buffer")
# sender reads ACK/NACK
print("\n SENDER CHECKING RESPONSE")
response = receiver_buffer[0]
number = receiver_buffer[1]
if response == "ACK":
    print("ACK received:", number)
    start = number + 1
else:
    print("NACK received:", number)
    start = number
print("TRANSMISSION COMPLETED")
```

Output:

Enter window size : 3
Enter message : Hello

SENDER

Frame NO : 0 DATA : H
Frame NO : 1 DATA : e
Frame NO : 2 DATA : l
Sender Buffer:
[C0,'H'), C1,'e'), C2,'l')]
Waiting for receiver...

RECEIVER

Received Frame : 0 DATA : H
Received Frame : 1 DATA : e
Received Frame : 2 DATA : l
Received Buffer:
C'ACK', 2)
SENDER CHECKING RESPONSE
ACK Received : 2

SENDER :

Frame no: 3 DATA : l
Frame no: 4 DATA : o
Sender Buffer :
[C3,'l'), C4,'o')]
waiting for receiver...

RECEIVER

Received frame : 3, Data : l
Received frame : 4, Data : o
Received Buffer :
C'ACK', 4)
SENDER CHECKING RESPONSE
ACK RECEIVED : 4

TRANSMISSION COMPLETED

Result:

Sliding window Protocols are learned and executed successfully.

28/3/24

Ex no: 10 Implementation of subnetting in
6/16/26 CISCO Packet Tracer
Simulator

Aim:

To implement subnetting using CISCO packet tracer and verifying communication between the subnets.

Subnetting:

It is the process of dividing large network into smaller logical networks called subnets. It helps in efficient utilization of IP address, reduces unnecessary network traffic & improve network management.

Requirements:

* cisco packet Tracer
* Routers
* Switches
* PCs
* Ethernet cables

Procedure:

1. open cisco packet Tracer
2. Create a new network topology
3. Add routers, switches, and PCs to the workspace, connect the device using ethernet cables.
4. Divide the network into smaller subnets using the 127 subnet mask.

5. Configure the IP address and subnet mask of the router interfaces.

6. Configure the IP address, subnet mask and default gateway for each PC.

7. Configure the required switch ports

8. Enable the router interfaces using the no shutdown command.

9. Check if combination of all devices uses the ping command to check connectivity between PCs.

10. Verify that the devices in the network communication.

step 1: Design the network Topology

→ Connect PC_0 and PC_1 to switch 0

→ Connect PC_2 and PC_3 to switch 1

step 2: Assign IP address

Device	IP address	subnet mask	Default gateway
PC_0	192.168.1.2	255.255.255.0	192.168.1.1
PC_1	192.168.1.3	255.255.255.0	192.168.1.1
PC_2	192.168.1.2	255.255.255.0	192.168.2.1
PC_3	192.168.2.3	255.255.255.0	192.168.2.1

step 3: Configure the route

→ open CLI

→ Terminal

enable configuration terminal

interface giganet Ethernet 0/0

ip address 192.168.1.1 255.255.255.0

no shutdown

exit

interface giganet Ethernet 0/1

ip address 192.168.1.1 255.255.255.0

no shutdown

exit

End

step 4: Test communication between same subnet

PC0, ping 192.168.1.3

check PC1 responds

step 5: Test communicatwin between different subnets

PC0, ping 192.168.2.2

check whether PC2 responds

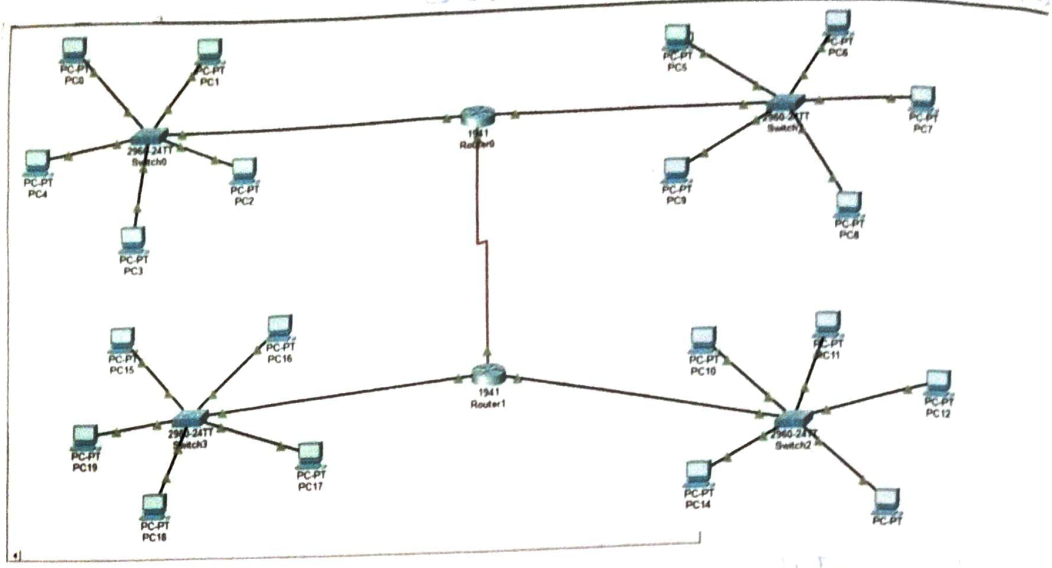
step 6: Test communication without router

→ disconnect router links

→ Attempt to ping PC2 from PC0 again

observation table

S.No	Source	Destination	Expected Result
1	PC0	PC1	Successful
2	PC0	PC2	Successful when router is configured & connected
3	PC0	PC2	Unsuccessful when the router connection between subnets is



Result :

Thus the subnetting was implemented successfully using cisco packet Tracer.

Ex no: 11 Implement your own Ping program

6/10/26

Aim:

To write a python program to implement the ping utility and check network connectivity between two hosts.

Algorithm:

1. Get destination IP address from user.
2. Send an ICMP echo request using the system ping command.
3. Receive the response from the destination
4. Display whether host is reachable.
5. Stop the program

Code:

```
import subprocess
import platform
ip = input("Enter destination IP address: ")
if platform.system().lower() == "windows":
    command = ["ping", "-n", "4", ip]
else:
    command = ["ping", "-c", "4", ip]
result = subprocess.run(command, capture_output=True, text=True)
print(result.stdout)
if result.returncode == 0:
    print("Host is reachable")
else:
    print("Host is unreachable")
```

Output:

I/P:

Enter destination IP address:

8.8.8.8

O/P:

Reply from 8.8.8.8

Host is reachable

Result:

Thus, the python program to check network connectivity using ping utility was executed successfully.

Ex no: 12 Packet sniffing using Raw sockets

13/10/26

Aim:

To write a python program using raw sockets to capture network packets and display their source IP, destination IP and protocol.

Algorithm:

1. Create raw sockets
2. Capture "incoming IP packets.
3. Extract IP header from each packet
4. Identify source IP address and destination IP address.
5. Identify protocol used
6. Display packet information

Code

```
import socket
import struct
try:
    sniffer = socket.socket(socket.AF_INET,
                            socket.SOCK_RAW,
                            socket.IPPROTO_IP)

except OSError as e:
    print("socket creation failed:", e)
    print("Run with administrator/root privileges.")
    raise SystemExit
print("packet sniffer started. Press Ctrl+C to stop")
```

```
try:
    while True:
        packet, address = sniffer.recvfrom(65535)
    if len(packet) < 20:
        continue
    version_ihl = packet[0]
    version = version_ihl >> 4
    ihl = (version_ihl & 0x0F) * 4
    if version != 4 or len(packet) < ihl
    continue
    ttl = packet[8]
    protocol = packet[9]
    src_ip = socket.inet_ntoa(packet[12:16])
    dst_ip = socket.inet_ntoa(packet[16:20])
    protocols = { 1: "ICMP"
                  6: "TCP"
                  17: "UDP"
                }
    Print("\n Packet captured")
    print("Source IP: ", src_ip)
    Print("Destination IP: ", dst_ip)
    print("Protocol: ", protocols.get(protocol, str(protocol)))
    print("TTL: ", ttl)
    print("Packet size: ", len(packet))
except KeyboardInterrupt:
    Print("\n Packet sniffing stopped.")
finally:
```

Output:

Packet sniffer started.
Press ctrl+C to stop

Packet captured
source IP : 192.168.1.10
Destination IP : 192.168.1.1
Protocol : TCP
TTL : 64
Packet size : 60

Result:

Thus, the python program using raw sockets was implemented to capture network packets executed successfully.

Ex no: 13
13/10/24

Internetworking with routers in cisco packet Tracer

Aim:

To design and configure a simple network using a router in cisco packet tracer and test connectivity between two PC's.

Procedure:

step 1: configuring router 1

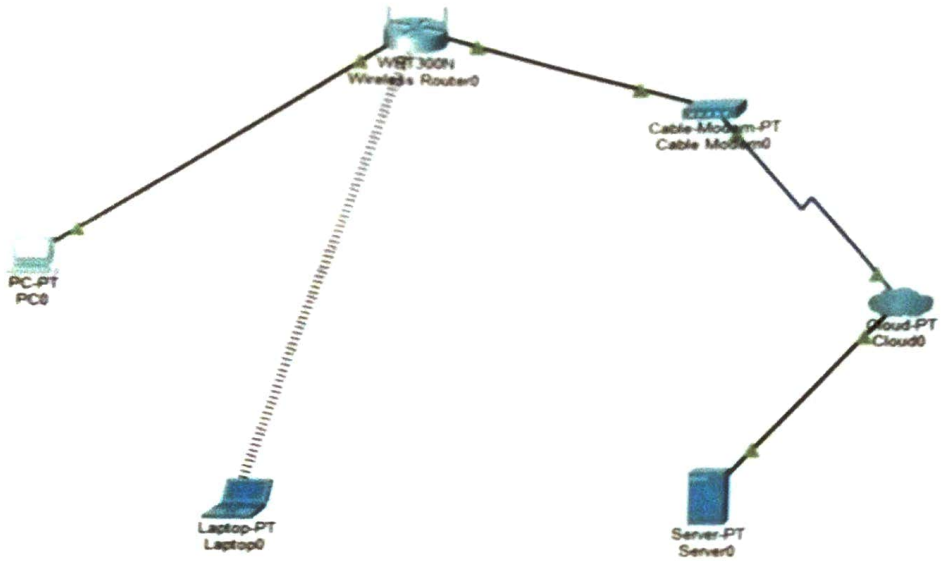
* Select the router and open CLI.
* Press enter to start configure router 1.
* Type enable to activate the priviledged mode.

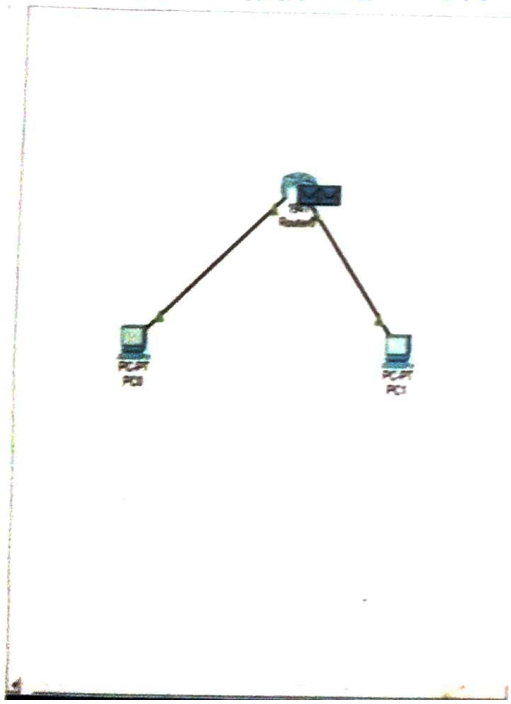
step 2:

* Assign IP address to every PC in the network.
* Select the PC, Go to the desktop and select IP configuration and assign an IP address, default gateway and subnet mask.
* Assign default gateway for both PC's.

step 3:

* Connect Fast ethernet for both PC's through copper straight-through cable.
* Connect fast ethernet router using a copper straight through cable.


WRT300N
Wireless Router0
Cable-Modem-PT
PC-PT
PC0
Cloud-PT
Cloud0
Laptop-PT
Laptop0
Server-PT
Server0


PC-PT
PC0
PC-PT
PC1

Exno: 14
13/10/26

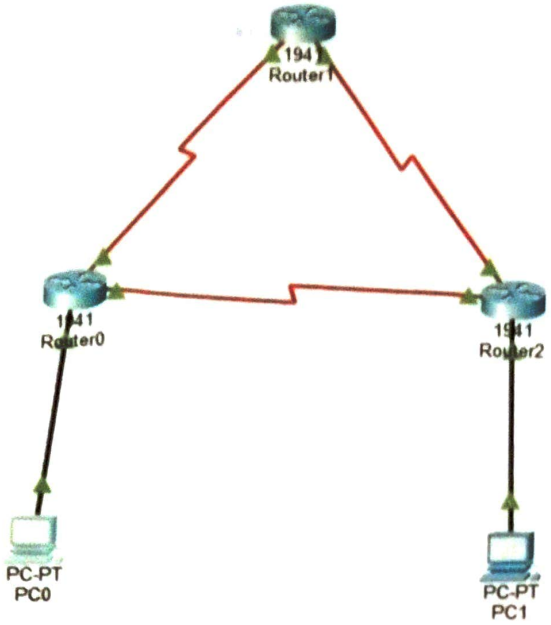
Routing Information Protocol

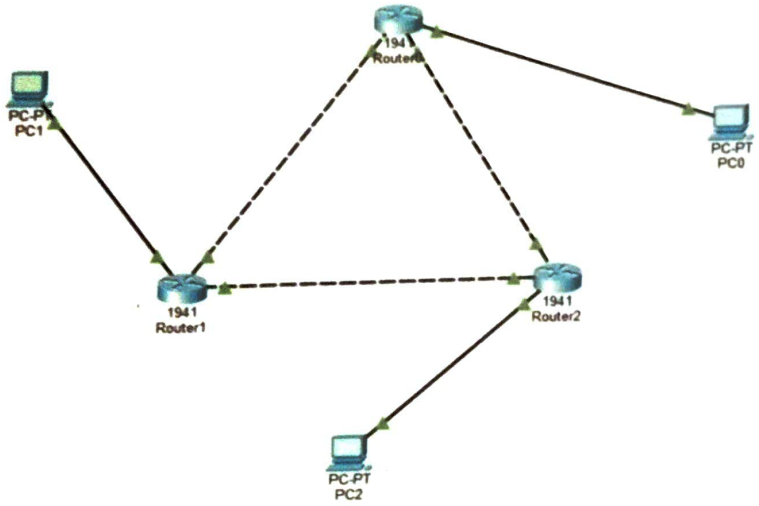
Aim:

To simulate a static routing configuration using cisco packet Tracer.

Procedure:

* Open cisco packet tracer and create the given network topology.
* Connect a 3 routers and PC's using suitable cables.
* Configure IP addresses for routers and PC's.
* Enable all routers interface using no shutdown.
* Configure static routers on router 0, router 1 and router 2 using IP mobile.
* Set main and backup router using different administrative distance value.
* Test connectivity
* Use tracer to verify the routing path.
* thus a static routing is successfully configured and verified.


1941
Router1
1941
Router0
1941
Router2
PC-PT
PC0
PC-PT
PC1


1941
Router0
PC-PT
PC1
PC-PT
PC0
1941
Router1
1941
Router2
PC-PT
PC2

```
Router>enable
Router#show ip route static
S    30.0.0.0/8 [20/0] via 40.0.0.2
S    50.0.0.0/8 [10/0] via 40.0.0.2
S    60.0.0.0/8 [1/0] via 40.0.0.2

Router#
%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0, changed state to do

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0, changed state to up

Router#show ip route static
S    30.0.0.0/8 [10/0] via 20.0.0.2
S    50.0.0.0/8 [10/0] via 40.0.0.2
S    60.0.0.0/8 [1/0] via 40.0.0.2
```

Result:

Routing Information Protocol succe
configured and dynamic routing betwee
and PC1 was verified.